

Simulación de Algoritmos de Planificación de CPU: FCFS, SJF y SRTF

Juan Diego Grajales Castillo
20221020128

Steven Navarro Parrales
20221020048

Alejandro Núñez Barrera
20231020139

26 de marzo de 2026

1. Introducción

La gestión de procesos es una de las funciones más importantes de un sistema operativo. Cuando múltiples procesos compiten por el uso del procesador, el sistema debe decidir cuál proceso se ejecuta en cada instante mediante algoritmos de planificación.

En este laboratorio se desarrolló un simulador en JavaScript que permite analizar el comportamiento de los algoritmos FCFS, SJF y SRTF, así como calcular métricas de rendimiento del sistema.

2. Objetivo

Reconocer la importancia de la gestión de procesos en un sistema operativo, especialmente en los algoritmos que controlan el turno justo en la ejecución de las tareas del sistema.

3. Algoritmos de Planificación

Los algoritmos de planificación determinan el orden en el que los procesos acceden al procesador. Cada algoritmo tiene un comportamiento diferente y afecta directamente las métricas del sistema como el tiempo de espera, el tiempo de retorno y la utilización del CPU. En este simulador se implementaron tres algoritmos clásicos ampliamente utilizados en el estudio de sistemas operativos.

3.1. FCFS (First Come First Served)

El algoritmo First Come First Served ejecuta los procesos en el mismo orden en que llegan al sistema. Este enfoque es equivalente a una cola FIFO, donde el primer proceso en entrar es el primero en ser atendido. Este algoritmo no es apropiativo, lo que significa que una vez que un proceso obtiene el CPU, lo mantiene hasta que finaliza su ejecución.

Una de las principales ventajas de FCFS es su simplicidad, ya que no requiere cálculos adicionales ni estimaciones del tiempo de ejecución. Sin embargo, puede generar el denominado efecto convoy, donde procesos cortos deben esperar a que procesos largos terminen, aumentando significativamente el tiempo promedio de espera.

En el simulador, este algoritmo se implementa seleccionando el primer proceso de la cola de listos.

Listing 1: Selección FCFS

```
1 if (this.algoritmo === "FCFS") {  
2     proceso = this.colaListos.shift();  
3 }
```

3.2. SJF (Shortest Job First)

El algoritmo Shortest Job First selecciona el proceso con el menor tiempo de ejecución estimado. Este algoritmo busca minimizar el tiempo promedio de espera, dando prioridad a los procesos más cortos. A diferencia de FCFS, el orden de ejecución no depende del momento de llegada sino de la duración del proceso.

SJF no es apropiativo, por lo que una vez que un proceso comienza su ejecución, no puede ser interrumpido hasta finalizar. Este algoritmo es óptimo

en términos de tiempo de espera promedio, pero requiere conocer o estimar previamente el tiempo de ejecución de cada proceso.

En el simulador, la implementación consiste en ordenar la cola de listos según el tiempo restante y seleccionar el proceso más corto.

Listing 2: Selección SJF

```
1 if (this.algoritmo === "SJF") {
2     this.colaListos.sort((a, b) =>
3         a.tiempoRestante - b.tiempoRestante
4     );
5     proceso = this.colaListos.shift();
6 }
```

3.3. SRTF (Shortest Remaining Time First)

El algoritmo Shortest Remaining Time First es la versión apropiativa del algoritmo SJF. En este caso, el planificador evalúa continuamente el tiempo restante de los procesos y puede interrumpir la ejecución actual si llega un proceso con menor tiempo restante.

Este algoritmo mejora el tiempo de respuesta y reduce el tiempo promedio de espera, aunque introduce mayor complejidad debido a los cambios de contexto. Cada vez que llega un nuevo proceso, el sistema debe verificar si este tiene menor tiempo restante que el proceso en ejecución.

En el simulador, esta lógica se implementa comparando el proceso actual con el proceso más corto en la cola de listos. Si el nuevo proceso tiene menor tiempo restante, se realiza la apropiación del CPU.

Listing 3: Selección SRTF

```
1 if (this.algoritmo === "SRTF") {
2     this.colaListos.sort((a, b) =>
3         a.tiempoRestante - b.tiempoRestante
4     );
5
6     if (this.enEjecucion &&
7         this.enEjecucion.tiempoRestante >
8         this.colaListos[0].tiempoRestante) {
9
10        this.colaListos.push(this.enEjecucion);
11        this.enEjecucion = this.colaListos.shift();
12    }
```

```
12     }  
13 }
```

4. Representación de los Procesos

Para la simulación de los algoritmos de planificación, cada proceso se representa mediante una estructura que almacena la información necesaria para su ejecución y para el cálculo de las métricas de rendimiento. Esta representación permite gestionar los procesos dentro de las colas de planificación y simular su paso por el CPU.

Cada proceso incluye atributos básicos como el identificador, el tiempo de llegada y el tiempo de ejecución. Además, se define el tiempo restante, el cual se actualiza dinámicamente durante la simulación, especialmente en el algoritmo SRTF donde los procesos pueden ser interrumpidos y reanudados posteriormente.

También se registran valores adicionales como el tiempo de inicio y finalización, los cuales permiten calcular métricas como el tiempo de retorno, tiempo de espera y tiempo de respuesta al finalizar la ejecución.

A continuación se muestra un fragmento de la estructura utilizada para representar los procesos en el simulador:

Listing 4: Representación de un proceso

```
1 const proceso = {  
2     id: id,  
3     llegada: llegada,  
4     duracion: duracion,  
5     tiempoRestante: duracion,  
6     inicio: null,  
7     fin: null  
8 };
```

5. Cálculo de Métricas de Planificación

Para evaluar el desempeño de los algoritmos de planificación implementados, el simulador calcula diferentes métricas que permiten analizar el com-

portamiento del sistema. Estas métricas reflejan la eficiencia en el uso del CPU y el impacto que cada algoritmo tiene sobre los procesos.

Cada proceso registra su tiempo de llegada, inicio de ejecución y finalización, lo que permite calcular los indicadores de rendimiento una vez termina la simulación.

5.1. Tiempo de Retorno

El tiempo de retorno representa el tiempo total que un proceso permanece dentro del sistema, desde su llegada hasta su finalización. Esta métrica permite medir cuánto tarda el sistema en completar una tarea.

$$T_{\text{retorno}} = T_{\text{finalizacion}} - T_{\text{llegada}}$$

Un tiempo de retorno bajo indica que el algoritmo de planificación permite finalizar los procesos rápidamente. Los algoritmos SJF y SRTF suelen obtener mejores resultados en esta métrica.

5.2. Tiempo de Espera

El tiempo de espera corresponde al tiempo que un proceso permanece en la cola de listos sin ser ejecutado. Esta métrica es importante porque refleja el grado de equidad del algoritmo.

$$T_{\text{espera}} = T_{\text{retorno}} - T_{\text{ejecucion}}$$

Un tiempo de espera elevado indica que los procesos pasan demasiado tiempo sin acceso al CPU. En este caso, el algoritmo FCFS suele presentar los valores más altos debido al efecto convoy.

5.3. Tiempo de Respuesta

El tiempo de respuesta mide el intervalo entre la llegada del proceso y su primera ejecución en el CPU. Esta métrica es especialmente importante en sistemas interactivos donde la rapidez de respuesta es fundamental.

$$T_{\text{respuesta}} = T_{\text{primerCPU}} - T_{\text{llegada}}$$

El algoritmo SRTF suele presentar mejores tiempos de respuesta, ya que permite interrumpir la ejecución para atender procesos más cortos.

5.4. Tiempo Perdido

El tiempo perdido representa el tiempo total en el que el proceso no está ejecutándose, incluyendo espera y posibles interrupciones.

$$T_{perdido} = T_{retorno} - T_{ejecucion}$$

Esta métrica permite identificar la eficiencia del algoritmo en el uso del CPU. Valores altos indican que los procesos permanecen inactivos durante largos periodos.

5.5. Penalidad

La penalidad es una métrica relativa que compara el tiempo total en el sistema con el tiempo real de ejecución del proceso.

$$Penalidad = \frac{T_{retorno}}{T_{ejecucion}}$$

Una penalidad cercana a 1 indica que el proceso se ejecutó casi inmediatamente, mientras que valores altos reflejan mayor tiempo de espera.

5.6. Análisis General de las Métricas

El análisis conjunto de estas métricas permite comparar el comportamiento de los algoritmos de planificación:

- FCFS presenta mayor tiempo de espera y penalidad
- SJF reduce el tiempo promedio de retorno
- SRTF mejora el tiempo de respuesta
- Los algoritmos apropiativos generan menor tiempo perdido

Estas métricas son calculadas automáticamente por el simulador y mostradas en una tabla comparativa, permitiendo evaluar el rendimiento de cada algoritmo durante la ejecución.

6. Resultados

Para evaluar el comportamiento de los algoritmos de planificación implementados, se ejecutó la simulación con un conjunto de procesos con diferentes tiempos de llegada y duración. A partir de la ejecución se calcularon las métricas de rendimiento para cada algoritmo.

Algoritmo	Retorno Prom.	Espera Prom.	Respuesta Prom.	Penalidad	Tipo
FCFS	Alto	Alto	Medio	Alta	No apropiativo
SJF	Medio	Bajo	Medio	Media	No apropiativo
SRTF	Bajo	Muy bajo	Bajo	Baja	Apropiativo

A partir de los resultados obtenidos se pueden observar varias diferencias entre los algoritmos. El algoritmo FCFS presenta el mayor tiempo de espera promedio debido a que ejecuta los procesos en orden de llegada sin considerar su duración. Esto genera que procesos cortos deban esperar a que procesos largos finalicen.

El algoritmo SJF mejora significativamente el tiempo de espera promedio, ya que prioriza los procesos con menor duración. Sin embargo, al no ser apropiativo, puede generar tiempos de respuesta mayores cuando aparece un proceso corto mientras otro proceso largo se encuentra en ejecución.

Por otro lado, el algoritmo SRTF presenta el mejor desempeño general. Al permitir la apropiación del CPU, los procesos cortos pueden ejecutarse rápidamente, lo que reduce tanto el tiempo de respuesta como la penalidad. No obstante, este algoritmo implica una mayor cantidad de cambios de contexto.

7. Conclusiones

El simulador permitió comprender la importancia de los algoritmos de planificación en la gestión de procesos. Se observó que FCFS es sencillo pero menos eficiente, mientras que SJF reduce el tiempo promedio de espera. SRTF proporciona el mejor tiempo de respuesta, aunque aumenta la complejidad del sistema.

El uso del simulador facilita el análisis comparativo entre algoritmos y permite evaluar métricas importantes para el diseño de sistemas operativos.